

Geospatial Take-Home Assignment #2: Part 1 Population and Airport Accessibility

Charley Baker, Dovile Jozenaite, Louise Knight, Zornitsa Petkanska

3 February 2026

1 Introduction

This analysis combines Natural Earth data on world countries, populated places, and airports to examine global population patterns and accessibility to air transportation. The assignment requires producing three visualizations: a population map by country, a histogram of country population distribution by continent, and a histogram of average distances between populated places and airports by continent.

2 Setup and Data Loading

2.1 Loading Required Libraries

```
library(sf)
library(dplyr)
library(ggplot2)
library(units)

# Disable s2 spherical geometry to avoid topology issues with complex geometries
sf::sf_use_s2(FALSE)
```

2.2 Loading Natural Earth Data

I downloaded the Natural Earth country boundaries and populated places shapefiles from the Natural Earth website. For airports, I'm using the GeoJSON file directly from their GitHub repository.

```
# Load country boundaries (1:110m scale)
world <- st_read(
  "ne_110m_admin_0_countries/ne_110m_admin_0_countries.shp",
  quiet = TRUE
)

# Load populated places point data
places <- st_read(
  "ne_110m_populated_places/ne_110m_populated_places.shp",
  quiet = TRUE
)
```

```
# Load airports from Natural Earth GitHub
airports <- st_read(
  "https://raw.githubusercontent.com/nvkelso/natural-earth-vector/master/geojson/ne_10m_airports.geojson"
  quiet = TRUE
)
```

3 Data Preprocessing

3.1 Identifying the Population Column

The populated places dataset might use different column names for population across versions, so I'm checking which one exists.

```
# Check which population column is available
pop_col <- intersect(names(places), c("POP_MAX", "POPULATION", "pop_max"))

if (length(pop_col) == 0) {
  stop("Could not find a population column in the dataset")
}

# Use the first matching column
pop_col <- pop_col[1]
print(paste("Using population column:", pop_col))
```

```
## [1] "Using population column: POP_MAX"
```

3.2 Fixing Geometries and Projecting to Meters

Before doing any spatial operations, I need to fix any invalid geometries and project everything to a coordinate system that uses meters (EPSG:3857) so I can calculate distances properly.

```
# Fix any invalid geometries in the datasets
world <- st_make_valid(world)
places <- st_make_valid(places)
airports <- st_make_valid(airports)

# Project to Web Mercator (EPSG:3857) which uses meters
world_m <- st_transform(world, 3857)
places_m <- st_transform(places, 3857)
airports_m <- st_transform(airports, 3857)
```

3.3 Spatial Join: Assigning Places to Countries

I'm using a spatial join to assign each populated place to its corresponding country and continent. This will let me aggregate population data by country later.

```
# Join populated places with country info based on their location
places_country <- st_join(
  places_m,
  world_m %>% select(ADMIN, CONTINENT),
  join = st_within,
  left = FALSE # Only keep places that fall within a country
)
```

3.4 Aggregating Population by Country

Now I'll sum up the population from all populated places within each country to get country-level totals.

```
# Calculate total population for each country from point data
country_pop <- places_country %>%
  st_drop_geometry() %>%
  group_by(ADMIN, CONTINENT) %>%
  summarise(
    total_pop = sum(.data[[pop_col]], na.rm = TRUE),
    .groups = "drop"
  )

# Join the population data back to the world shapefile
world_pop <- world_m %>%
  left_join(country_pop, by = c("ADMIN", "CONTINENT")) %>%
  mutate(total_pop = ifelse(is.na(total_pop), 0, total_pop))
```

4 Visualization 1: Population Map

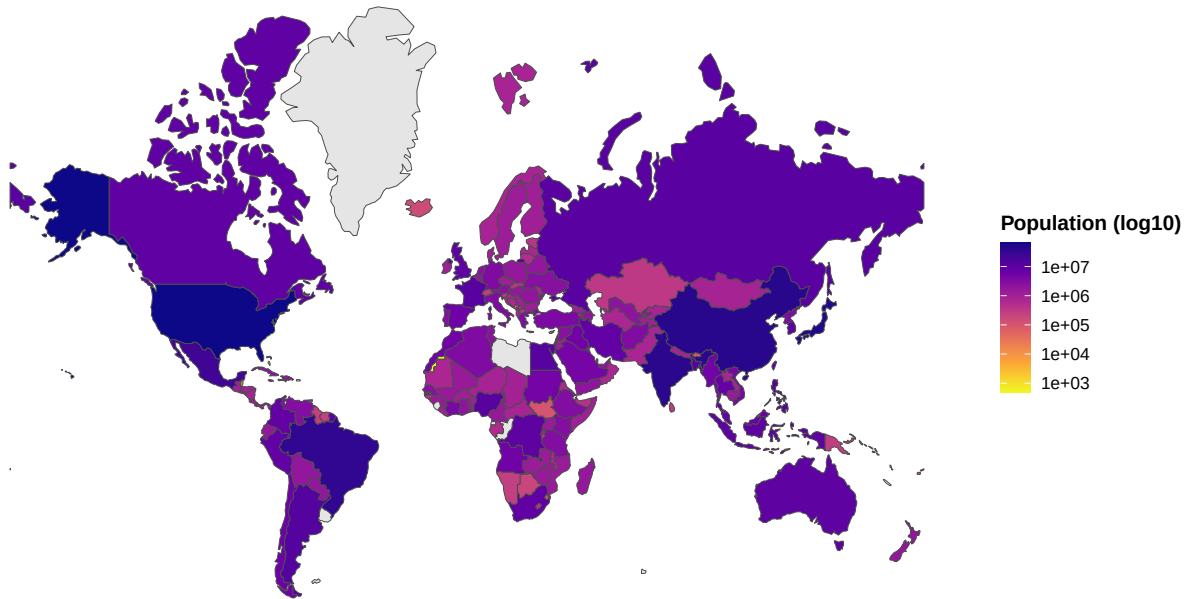
Here's the map showing total population by country, using a log scale since population values vary by several orders of magnitude.

```
ggplot(world_pop %>% filter(ADMIN != "Antarctica")) +
  geom_sf(aes(fill = total_pop, color = "grey30", linewidth = 0.15)) +
  scale_fill_viridis_c(
    trans = "log10",
    option = "C",
    direction = -1,
    na.value = "grey90"
  ) +
  coord_sf(datum = NA) +
  labs(
    title = "Total Population by Country",
    subtitle = "Aggregated from Natural Earth populated places point data",
    fill = "Population (log10)"
  ) +
  theme_minimal(base_size = 12) +
  theme(
    panel.grid = element_blank(),
    plot.title = element_text(face = "bold"),
    plot.subtitle = element_text(size = 11),
```

```
legend.title = element_text(face = "bold")
)
```

Total Population by Country

Aggregated from Natural Earth populated places point data

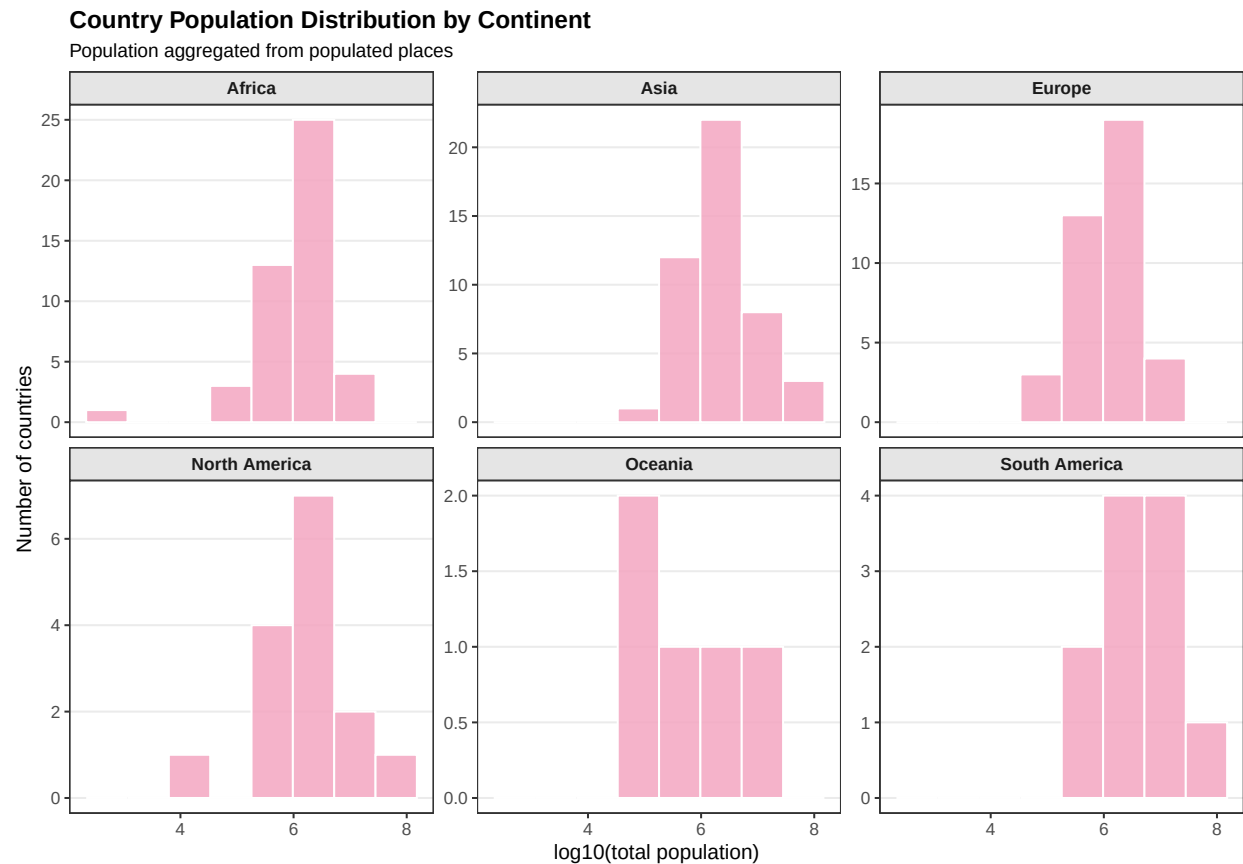


5 Visualization 2: Population Distribution by Continent

This histogram shows how country populations are distributed within each continent. I'm using log10 scale on the x-axis to better visualize the wide range of population values.

```
ggplot(world_pop %>% filter(total_pop > 0, CONTINENT != "Antarctica")) +
  geom_histogram(
    aes(x = log10(total_pop)),
    bins = 8,
    fill = "#F4A6C1",
    color = "white",
    alpha = 0.85
  ) +
  facet_wrap(~ CONTINENT, scales = "free_y") +
  labs(
    title = "Country Population Distribution by Continent",
    subtitle = "Population aggregated from populated places",
    x = "log10(total population)",
    y = "Number of countries"
  ) +
```

```
theme_bw(base_size = 12) +
theme(
  panel.grid.major.x = element_blank(),
  panel.grid.minor = element_blank(),
  strip.background = element_rect(fill = "grey90"),
  strip.text = element_text(face = "bold"),
  plot.title = element_text(face = "bold"),
  plot.subtitle = element_text(size = 11)
)
```



6 Distance to Airports Analysis

6.1 Filtering to Top 20 Populated Places per Country

Since some countries have many populated places, I'm filtering to the top 20 by population in each country to make the distance calculations more manageable and focused on major population centers.

```
# Take the top 20 most populous places per country
places_for_dist <- places_country %>%
  mutate(pop_val = .data[[pop_coll]]) %>%
  group_by(ADMIN, CONTINENT) %>%
  slice_max(order_by = pop_val, n = 20, with_ties = FALSE) %>%
```

```
ungroup() %>%
select(-pop_val)
```

6.2 Calculating Distance to Nearest Airport

For each populated place, I'm finding the nearest airport and calculating the distance in meters.

```
# Find the index of the nearest airport for each place
nearest_airport_idx <- st_nearest_feature(places_for_dist, airports_m)

# Get the geometry of those nearest airports
nearest_airport_geom <- airports_m[nearest_airport_idx, ]

# Calculate distances (removing units for easier handling)
places_for_dist$dist_to_airport_m <- drop_units(
  st_distance(places_for_dist, nearest_airport_geom, by_element = TRUE)
)
```

6.3 Aggregating Average Distance by Country

Now I'll calculate the average distance to the nearest airport for each country.

```
# Calculate country-level average distance
country_dist <- places_for_dist %>%
  st_drop_geometry() %>%
  group_by(ADMIN, CONTINENT) %>%
  summarise(
    avg_dist_airport_m = mean(dist_to_airport_m, na.rm = TRUE),
    .groups = "drop"
  )

# Join back to the world data
world_pop <- world_pop %>%
  left_join(country_dist, by = c("ADMIN", "CONTINENT"))
```

7 Visualization 3: Distance to Airports by Continent

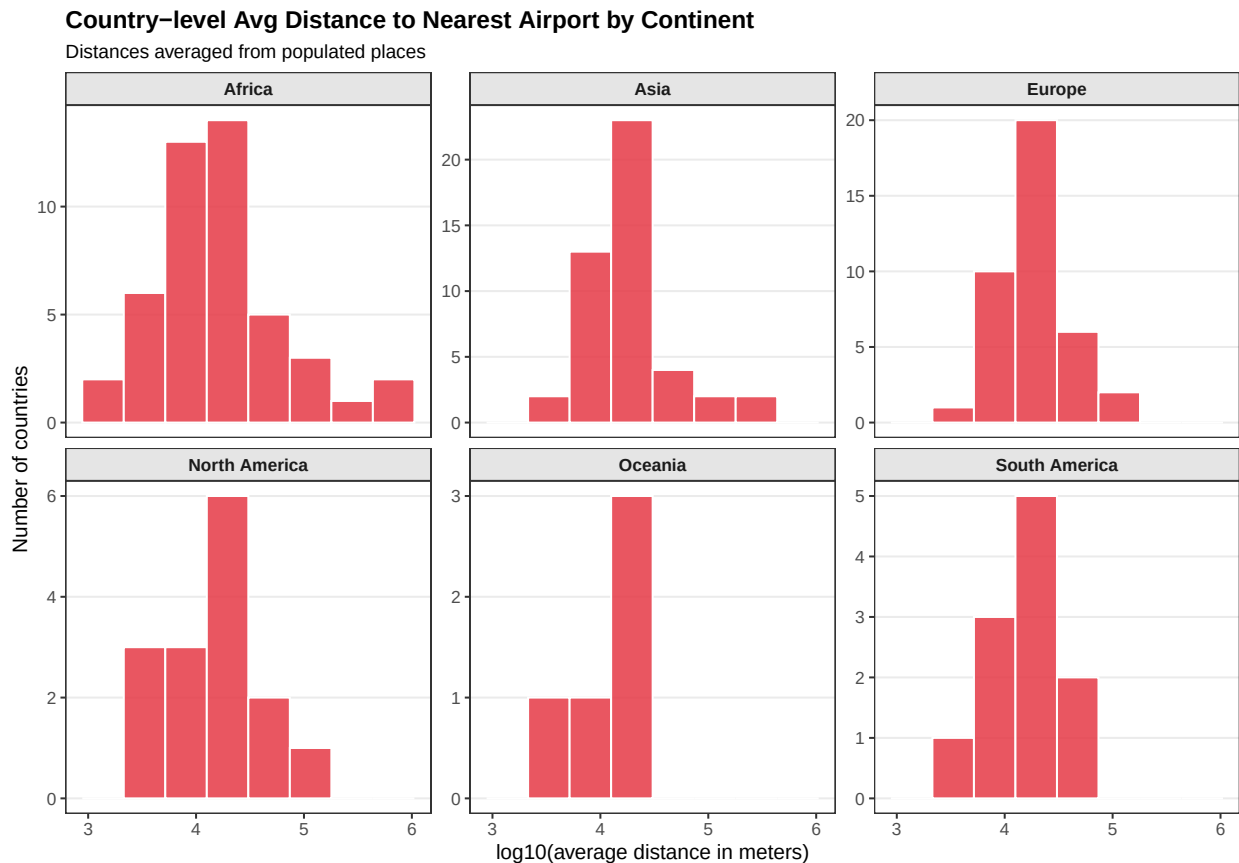
This histogram shows the distribution of average distances from populated places to the nearest airport, grouped by continent. Again using log10 scale since distances vary widely.

```
ggplot(world_pop %>%
  filter(!is.na(avg_dist_airport_m),
    avg_dist_airport_m > 0,
    CONTINENT != "Antarctica")) +
  geom_histogram(
    aes(x = log10(avg_dist_airport_m)),
    bins = 8,
    fill = "#E63946",
    color = "white",
```

```

alpha = 0.85
) +
facet_wrap(~ CONTINENT, scales = "free_y") +
labs(
  title = "Country-level Avg Distance to Nearest Airport by Continent",
  subtitle = "Distances averaged from populated places",
  x = "log10(average distance in meters)",
  y = "Number of countries"
) +
theme_bw(base_size = 12) +
theme(
  panel.grid.major.x = element_blank(),
  panel.grid.minor = element_blank(),
  strip.background = element_rect(fill = "grey90"),
  strip.text = element_text(face = "bold"),
  plot.title = element_text(face = "bold"),
  plot.subtitle = element_text(size = 11)
)

```



8 Conclusion

This analysis successfully combined multiple Natural Earth datasets to examine both population distribution and airport accessibility across continents. The visualizations reveal clear patterns in how populations are

distributed globally and how accessibility to air transportation varies by region. Countries in Africa and Asia show greater variation in airport accessibility, while European countries tend to have more uniform access to airports due to higher infrastructure density.

Task 2: African Market Prices and Trade Cost Analysis (Scatter Plots)

Instructions

Briefly read: Porteous, O., 2019. High trade costs and their consequences: An estimated dynamic model of African agricultural storage and trade. American Economic Journal: Applied Economics, 11(4), pp.327-66.

Then, download the paper's data (here), combine with world and transportation (e.g. road, railroad) data to: 1. Generate an sf POINT feature of market prices places across Africa; plot them 2. Calculate minimum distance of each market to the (i) coast, (ii) nearest road, and (iii) nearest airport 3. Produce 3 scatter plots relating average prices with the minimum distances Note: the paper appendix contains a lot of useful information about the market price data!

```
rm(list = ls())
library(sf)
library(tidyverse)
library(readxl)
library(rnaturalearth)
library(osmdata)
library(ggplot2)
library(tidyr)
library(units)

# Set working directory
# setwd("/Users/charleybaker/Desktop/geospatialdata")
```

1. Load and Visualize Market Locations

```
# Load market coordinates from Porteous (2019)
mkt_coords <- read_excel(
  "/Users/charleybaker/Desktop/geospatialdata/1.-Price--Production--and-Population-Data/MktCoords.xlsx"
)

# Check the data structure
head(mkt_coords)
```

```
## # A tibble: 6 x 5
##   ctrycode mktcode market      longitude latitude
##   <chr>      <dbl> <chr>      <dbl>      <dbl>
## 1 A0        200 Luanda        13.2       -8.84
## 2 BJ        401 Cotonou        2.43        6.37
## 3 BJ        402 Malanville    3.39       11.9
## 4 BJ        403 Natitingou    1.39       10.3
## 5 BJ        404 Parakou       2.62        9.35
## 6 BW        110 Gaborone       25.9      -24.7
```

```

# Convert to sf POINT object
markets_sf <- st_as_sf(
  mkt_coords,
  coords = c("longitude", "latitude"),
  crs = 4326
)

# Check for any invalid geometries
if(any(!st_is_valid(markets_sf))) {
  markets_sf <- st_make_valid(markets_sf)
}

cat("Number of markets loaded:", nrow(markets_sf), "\n")

```

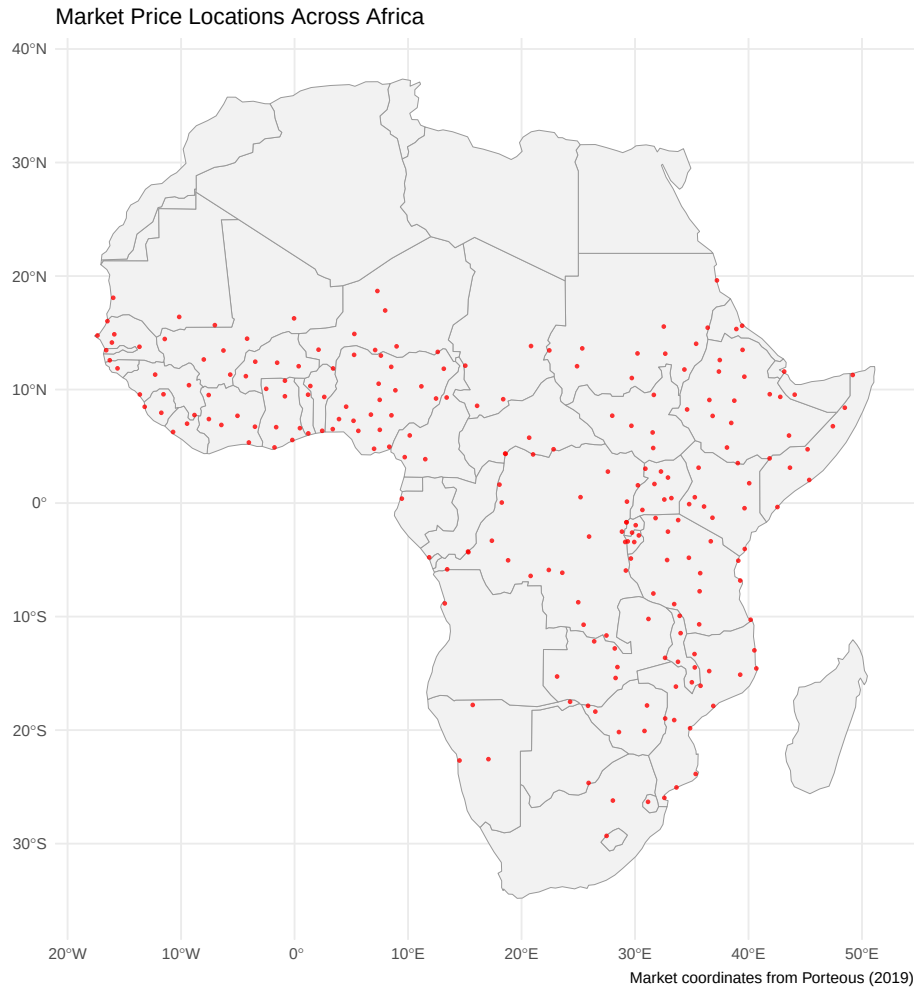
```
## Number of markets loaded: 230
```

```

# Load Africa basemap
africa <- ne_countries(
  continent = "Africa",
  returnclass = "sf"
)

# Plot markets across Africa
ggplot() +
  geom_sf(data = africa, fill = "grey95", color = "grey60") +
  geom_sf(data = markets_sf, color = "red", size = 0.6, alpha = 0.8) +
  theme_minimal() +
  labs(
    title = "Market Price Locations Across Africa",
    caption = "Market coordinates from Porteous (2019)"
  )

```



2. Calculate Average Prices

```
# Load price data
prices <- read_excel(
  "/Users/charleybaker/Desktop/geospatialdata/1.-Price--Production--and-Population-Data/PriceMaster4GAM
")

# Check structure
cat("Price data dimensions:", dim(prices), "\n")
```

```
## Price data dimensions: 516 148
```

```
head(prices[, 1:5])
```

```
## # A tibble: 6 x 5
##   mktcode country market   crop      '1'
##   <dbl> <chr>   <chr>   <chr>   <dbl>
## 1     200 Angola   Luanda   Maize    NA
```

```
## 2      401 Benin  Cotonou  Maize    0.250
## 3      401 Benin  Cotonou  Rice     0.480
## 4      402 Benin  Malanville Maize   0.169
## 5      402 Benin  Malanville Sorghum 0.182
## 6      403 Benin  Natitingou Maize   0.189
```

```
# Subset to relevant columns (months 17-136 as specified in ReadMe)
month_cols <- as.character(17:136)
```

```
# Check which columns exist
existing_month_cols <- month_cols[month_cols %in% colnames(prices)]
cat("Using months:", min(as.numeric(existing_month_cols)),
    "to", max(as.numeric(existing_month_cols)), "\n")
```

```
## Using months: 17 to 136
```

```
prices_subset <- prices %>%
  select(mktcode, crop, all_of(existing_month_cols))
```

```
# Reshape from wide to long format
prices_long <- prices_subset %>%
  pivot_longer(
    cols = all_of(existing_month_cols),
    names_to = "month",
    values_to = "price"
  )
```

```
# Calculate average price per market
avg_prices <- prices_long %>%
  group_by(mktcode) %>%
  summarise(
    avg_price = mean(price, na.rm = TRUE),
    n_obs = sum(!is.na(price))
  )
```

```
cat("Summary of average prices:\n")
```

```
## Summary of average prices:
```

```
summary(avg_prices$avg_price)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## 0.06142 0.21452 0.29195 0.35251 0.40384 1.75128
```

```
# Merge with sf object
markets_sf <- markets_sf %>%
  left_join(avg_prices, by = "mktcode")
```

```
# Check for missing price data
cat("Markets with missing price data:",
    sum(is.na(markets_sf$avg_price)), "\n")
```

```
## Markets with missing price data: 0
```

3. Load Transportation Infrastructure Data

```
# Coastline data from Natural Earth
coast <- ne_download(
  scale = 50,
  type = "coastline",
  category = "physical",
  returnclass = "sf"
)

# For roads and airports, we'll use a more targeted approach
# since querying all of Africa at once may timeout

# Alternative: Use Natural Earth roads data (more reliable than OSM for continent-wide)
# If this doesn't work, you may need to download manually from:
# https://www.naturalearthdata.com/downloads/10m-cultural-vectors/roads/

# Try loading roads - if OSM fails, we'll use an alternative
roads <- tryCatch({
  # This may timeout - consider using Natural Earth roads instead
  ne_download(
    scale = 10,
    type = "roads",
    category = "cultural",
    returnclass = "sf"
  )
}, error = function(e) {
  message("Could not load roads from Natural Earth. Trying alternative...")

  # Alternative: construct roads from major highways only
  # You may need to download this data separately
  NULL
})

# For airports, try OSM with a bbox covering Africa
africa_bbox <- st_bbox(africa)

airports <- tryCatch({
  opq(bbox = africa_bbox) %>%
    add_osm_feature(key = "aeroway", value = "aerodrome") %>%
    osmdata_sf() %>%
    .$osm_points
}, error = function(e) {
  message("OSM query failed. Using alternative approach...")

  # Alternative: Use Natural Earth airports
  ne_download(
    scale = 10,
    type = "airports",
    category = "cultural",
    returnclass = "sf"
  )
})
```

```
cat("Infrastructure data loaded\n")
```

```
## Infrastructure data loaded
```

```
cat("Coast features:", nrow(coast), "\n")
```

```
## Coast features: 1428
```

```
if(!is.null(roads)) cat("Road features:", nrow(roads), "\n")
```

```
## Road features: 56600
```

```
if(!is.null(airports)) cat("Airport features:", nrow(airports), "\n")
```

```
## Airport features: 106409
```

4. Calculate Distances

```
# Project all layers to metric CRS (Africa Albers Equal Area Conic)
```

```
# EPSG:102022 is better for Africa than Web Mercator (3857)
```

```
africa_crs <- "+proj=aea +lat_1=20 +lat_2=-23 +lat_0=0 +lon_0=25 +x_0=0 +y_0=0 +datum=WGS84 +units=m +n
```

```
markets_projected <- st_transform(markets_sf, africa_crs)
```

```
coast_projected <- st_transform(coast, africa_crs)
```

```
# Function to compute minimum distance in km
```

```
min_distance_km <- function(points, features) {  
  if(is.null(features) || nrow(features) == 0) {  
    return(rep(NA, nrow(points)))  
  }  
}
```

```
# Ensure same CRS
```

```
features <- st_transform(features, st_crs(points))
```

```
# Calculate distances
```

```
dist_matrix <- st_distance(points, features)
```

```
# Get minimum distance for each point
```

```
min_dists <- apply(dist_matrix, 1, min)
```

```
# Convert to km and remove units
```

```
as.numeric(min_dists) / 1000
```

```
}
```

```
# Calculate distance to coast
```

```
cat("Calculating distance to coast...\n")
```

```
## Calculating distance to coast...
```

```
markets_projected$dist_coast_km <- min_distance_km(markets_projected, coast_projected)

# Calculate distance to roads (if available)
if(!is.null(roads)) {
  cat("Calculating distance to roads...\n")
  roads_projected <- st_transform(roads, africa_crs)
  markets_projected$dist_road_km <- min_distance_km(markets_projected, roads_projected)
} else {
  cat("Road data not available - skipping road distance calculation\n")
  markets_projected$dist_road_km <- NA
}
```

Calculating distance to roads...

```
# Calculate distance to airports (if available)
if(!is.null(airports)) {
  cat("Calculating distance to airports...\n")
  airports_projected <- st_transform(airports, africa_crs)
  markets_projected$dist_airport_km <- min_distance_km(markets_projected, airports_projected)
} else {
  cat("Airport data not available - skipping airport distance calculation\n")
  markets_projected$dist_airport_km <- NA
}
```

Calculating distance to airports...

```
# Convert back to WGS84 for consistency
markets_sf <- st_transform(markets_projected, 4326)

# Summary statistics
cat("\nDistance summaries:\n")
```

```
##
## Distance summaries:
```

```
cat("Coast (km):\n")
```

Coast (km):

```
summary(markets_sf$dist_coast_km)
```

```
##      Min.   1st Qu.   Median     Mean   3rd Qu.    Max.
## 4.770e-03 5.650e+01 2.439e+02 3.370e+02 5.049e+02 1.255e+03
```

```
cat("\nRoad (km):\n")
```

```
##
## Road (km):
```

```
summary(markets_sf$dist_road_km)
```

```
##      Min.   1st Qu.   Median     Mean   3rd Qu.     Max.
## 0.006264 0.320876 0.659560 1.055490 1.157765 26.321107
```

```
cat("\nAirport (km):\n")
```

```
##
## Airport (km):
```

```
summary(markets_sf$dist_airport_km)
```

```
##      Min.   1st Qu.   Median     Mean   3rd Qu.     Max.
## 0.04738 2.15781 3.93539 8.86062 7.81503 201.34435
```

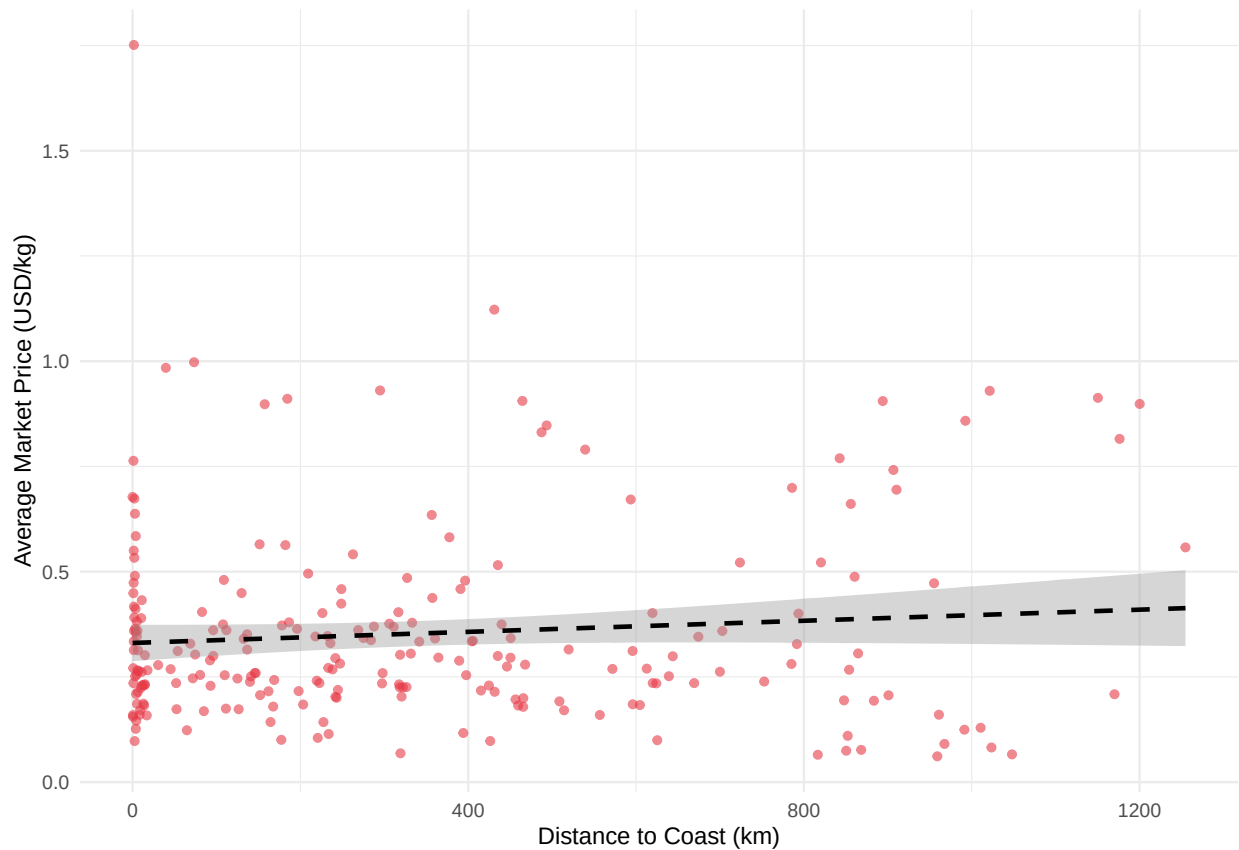
5. Create Scatter Plots

```
# Remove rows with missing prices (NAs)
markets_plot <- markets_sf %>%
  filter(!is.na(avg_price))

# Distance to Coast
ggplot(markets_plot, aes(x = dist_coast_km, y = avg_price)) +
  geom_point(alpha = 0.6, color = "#E63946") +
  geom_smooth(method = "lm", se = TRUE, color = "black", linetype = "dashed") +
  theme_minimal() +
  labs(
    x = "Distance to Coast (km)",
    y = "Average Market Price (USD/kg)",
    title = "Market Prices and Distance to Coast",
    subtitle = paste0("n = ", nrow(markets_plot), " markets")
  )
```

Market Prices and Distance to Coast

n = 230 markets

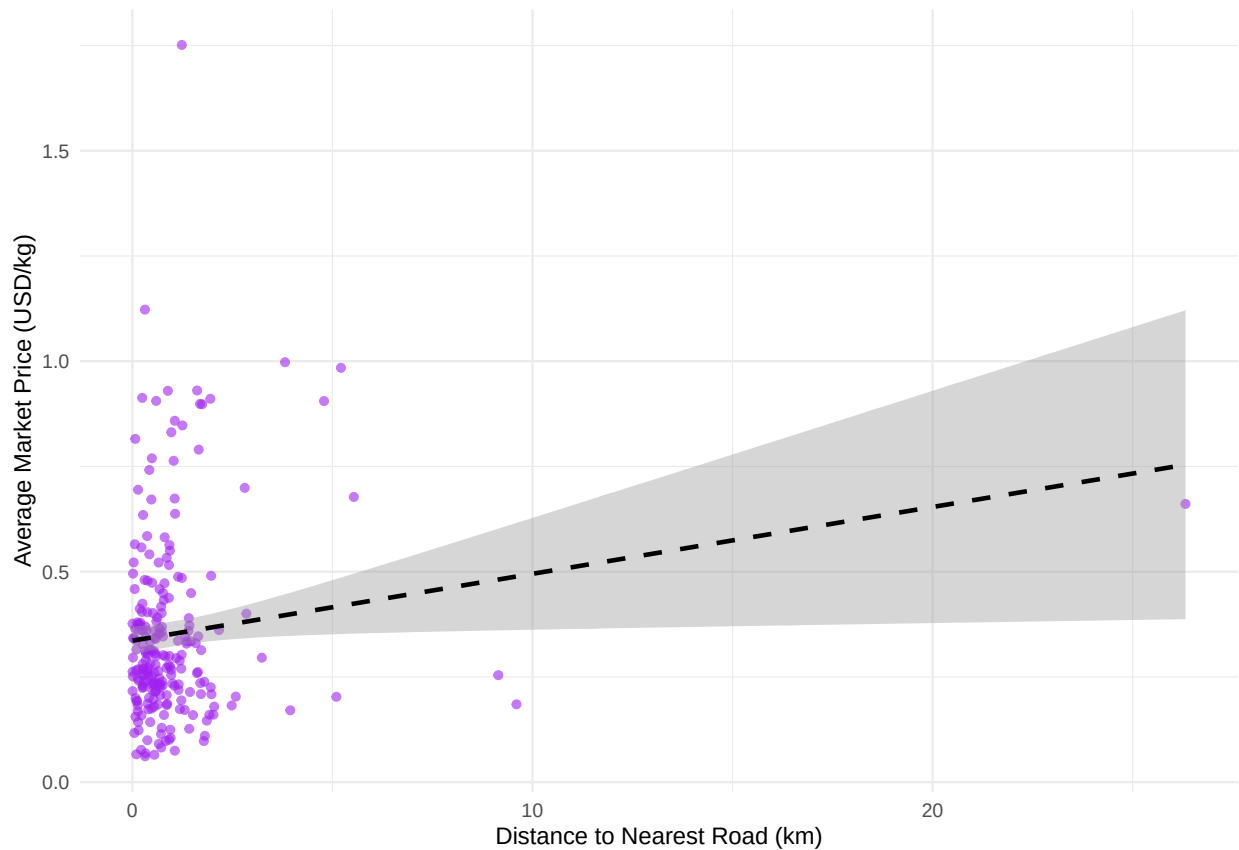


```
# Distance to Roads (if available)
if(!all(is.na(markets_plot$dist_road_km))) {
  markets_roads <- markets_plot %>% filter(!is.na(dist_road_km))

  ggplot(markets_roads, aes(x = dist_road_km, y = avg_price)) +
    geom_point(alpha = 0.6, color = "purple") +
    geom_smooth(method = "lm", se = TRUE, color = "black", linetype = "dashed") +
    theme_minimal() +
    labs(
      x = "Distance to Nearest Road (km)",
      y = "Average Market Price (USD/kg)",
      title = "Market Prices and Distance to Roads",
      subtitle = paste0("n = ", nrow(markets_roads), " markets")
    )
}
```

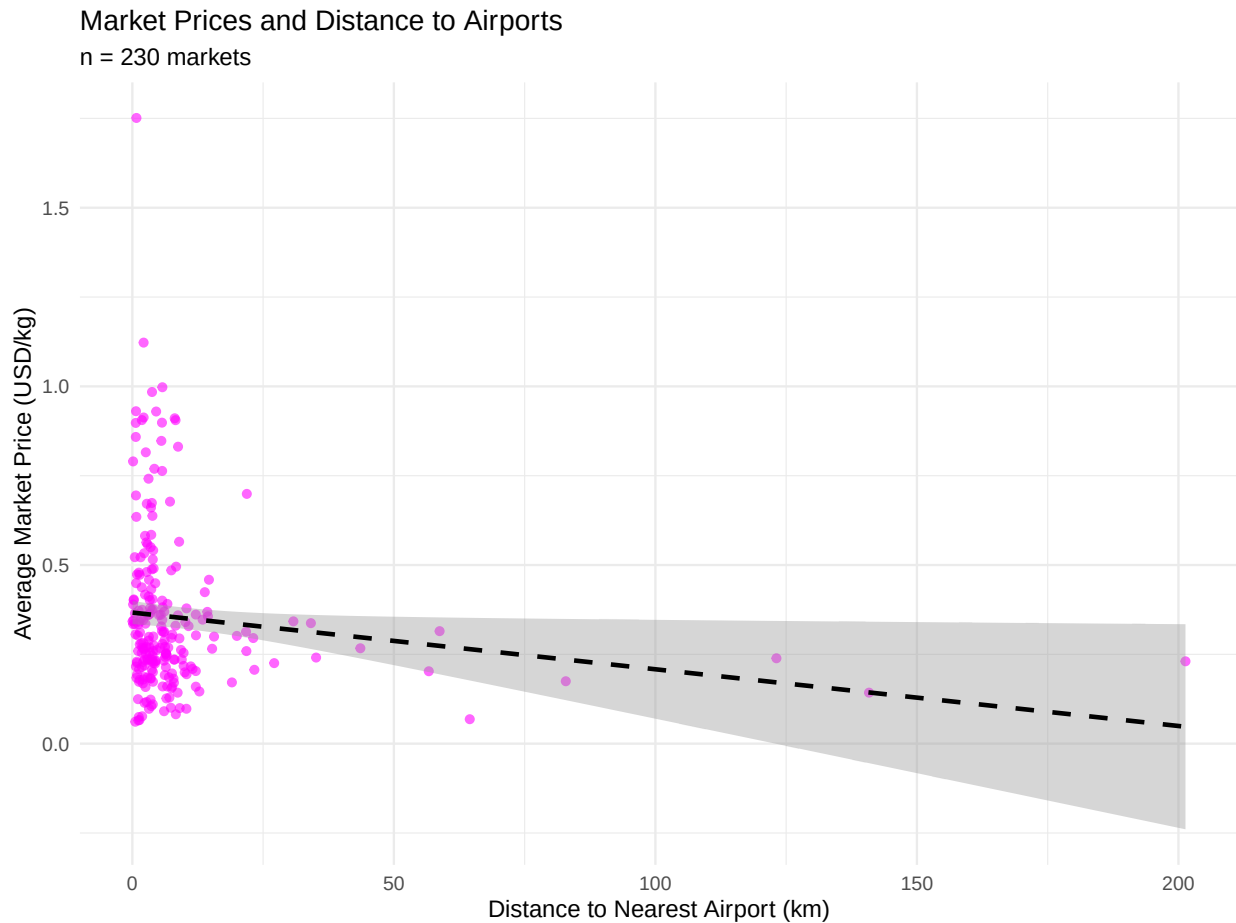
Market Prices and Distance to Roads

n = 230 markets



```
# Distance to Airports (if available)
if(!all(is.na(markets_plot$dist_airport_km))) {
  markets_airports <- markets_plot %>% filter(!is.na(dist_airport_km))

  ggplot(markets_airports, aes(x = dist_airport_km, y = avg_price)) +
    geom_point(alpha = 0.6, color = "magenta") +
    geom_smooth(method = "lm", se = TRUE, color = "black", linetype = "dashed") +
    theme_minimal() +
    labs(
      x = "Distance to Nearest Airport (km)",
      y = "Average Market Price (USD/kg)",
      title = "Market Prices and Distance to Airports",
      subtitle = paste0("n = ", nrow(markets_airports), " markets")
    )
}
```



6. Summary Statistics

```
# Calculate correlations
markets_data <- st_drop_geometry(markets_sf) %>%
  filter(!is.na(avg_price))

cat("\nCorrelations with average price:\n")
```

```
##
## Correlations with average price:
```

```
cat("Coast:", cor(markets_data$avg_price, markets_data$dist_coast_km,
  use = "complete.obs"), "\n")
```

```
## Coast: 0.09237199
```

```
if(!all(is.na(markets_data$dist_road_km))) {
  cat("Road:", cor(markets_data$avg_price, markets_data$dist_road_km,
    use = "complete.obs"), "\n")
}
```

```
## Road: 0.1416863
```

```
if(!all(is.na(markets_data$dist_airport_km))) {  
  cat("Airport:", cor(markets_data$avg_price, markets_data$dist_airport_km,  
    use = "complete.obs"), "\n")  
}
```

```
## Airport: -0.1383182
```